



Boss Bridge Protocol Audit Report

Version 1.0

Cyfrin.io

April 24, 2026

Boss Bridge Protocol Audit Report

vridhib

April 24, 2026

Prepared by: vridhib

Lead Auditors: vridhib

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues Found
- Findings
 - High
 - * [H-1] Users Who Give Tokens Approvals to `L1BossBridge` May Have Those Assets Stolen
 - * [H-2] Calling `L1BossBridge::depositTokensToL2` with the `from` and `to` Arguments Set to the Vault Contract Allows Infinite Minting of Unbacked Tokens
 - * [H-3] Lack of Replay Protection in `L1BossBridge::withdrawTokensToL1` Allows Withdrawals by Signature to be Replayed

- * [H-4] `L1BossBridge::sendToL1` Allowing Arbitrary Calls Enables Users to Call `L1Vault::approveTo` & Give Themselves Infinite Allowance of Vault Funds
 - * [H-5] Assembly `CREATE` Usage in `TokenFactory::deployToken` Is Incompatible with ZKsync Native EraVM
 - * [H-6] `L1BossBridge::DEPOSIT_LIMIT` Check in `L1BossBridge::depositTokensToL2` Combined with Front-Running Enables Permanent DoS of Deposits
 - * [H-7] Missing Validation in `L1BossBridge::withdrawTokensToL1` on the Withdrawal Amount Allows Attackers to Withdraw More Tokens Than Deposited
 - * [H-8] `TokenFactory::deployToken` Causes All Minted Tokens to Be Permanently Locked in the Factory Contract
- Low
- * [L-1] Missing Events on Withdrawal Functions Obscure Off-Chain Monitoring and Incident Response
 - * [L-2] `TokenFactory::deployToken` Can Create Multiple Tokens with the Same `symbol`

Protocol Summary

Boss Bridge is a simple cross-chain token bridge that moves an ERC20 token from an Ethereum L1 to a ZKsync L2. Users deposit tokens into a secure vault on L1. A successful deposit emits an event that an off-chain service picks up and triggers a corresponding token mint on L2. Withdrawals from the L2 back to the L1 require a signature from an authorized operator. The bridge includes several safety mechanisms: an owner who can pause the protocol in emergencies, a hard deposit limit to cap the amount of tokens locked, and an operator-approval flow for withdrawals. The protocol is intended to be deployed on both Ethereum Mainnet and ZKsync Era, with a factory that deploys new token contracts. The supported ERC20 tokens are restricted to the standard L1Token contract (and exact copies).

Disclaimer

The vridhib team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

The CodeHawks severity matrix was utilized to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash: [07af21653ab3e8a8362bf5f63eb058047f562375](#).

Scope

- ./src/L1BossBridge.sol
- ./src/L1Token.sol
- ./src/L1Vault.sol
- ./src/TokenFactory.sol

Roles

- **Bridge Owner:** A centralized bridge owner who can pause and unpause the bridge in the event of an emergency as well as set signers.
- **Signer:** Users who can “send” a token from the L2 to the L1.
- **Vault:** The contract owned by the bridge that holds the tokens.
- **Users:** Users mainly call `depositTokensToL2`, when they want to send tokens from the L1 to the L2.

Executive Summary

The Boss Bridge protocol was audited over a three-day period by one security researcher using a combination of manual review, static analysis (Slither and Aderyn), and fuzzing/invariant testing (Foundry). The audit primarily focused on deposit and withdrawal mechanics, signature validation, access controls, token factory logic, and cross-chain compatibility, as defined in the protocol summary.

Overall, the protocol contains several critical-and-high severity flaws that make it unsuitable for production deployment without significant remediation. The most severe issues allow an attacker to steal tokens from any user who has approved the bridge, drain the vault by making arbitrary calls via the `sendToL1` function, and replay withdrawal signatures indefinitely. Additionally, the deposit function lacks any `from` validation, enabling an attacker to siphon approved tokens from any account. The `sendToL1` function can be abused to call `approveTo` on the vault and give the attacker unlimited allowance over all vault funds. Withdrawal signatures have no replay protection, allowing a single valid signature to iteratively drain the vault. The deposit limit is hard-coded and can be front-run to permanently deny deposits. Furthermore, the withdrawal amount is not validated against prior deposits, allowing attackers to withdraw far more than they deposited. On another note (regarding deployment setup), the `TokenFactory` uses an assembly `CREATE` opcode that is incompatible with the native on ZKsync Era EraVM, breaking token deployment. Finally, the same factory locks all minted tokens inside itself because the token constructor mints to `msg.sender` (the factory), which has no token-handling functions.

The findings are summarized as follows:

- **8 High-Severity** issues:

- Anyone can steal approved tokens from users via `depositTokensToL2`.
- Depositing from the vault address enables infinite minting of unbacked tokens.
- Withdrawal signatures lack replay protection, allowing indefinite draining.
- The `sendToL1` arbitrary calls let an attacker approve themselves vault allowances to steal funds.
- Assembly `CREATE` in `TokenFactory` is incompatible with ZKsync EraVM.
- Immutable deposit limit enables permanent DoS of deposits via front-running.
- Missing withdrawal amount validation allows withdrawing more tokens than were deposited.
- `TokenFactory` locks all minted tokens in the factory contract forever.

- **2 Low-Severity** issues:

- Missing events on withdrawal functions hinder off-chain monitoring
- `TokenFactory` allows duplicate token symbols, overwriting existing addresses

Due to the severity and number of findings, the protocol should not be deployed in its current state. It is strongly recommended to redesign the deposit flow so that `from` must be `msg.sender` (or implement proper delegation), restrict the targets callable by `sendToL1`, add chain ID and nonce-based replay protection to withdrawals, replace the assembly `CREATE` with Solidity's `new` keyword, make the deposit limit adjustable, and validate withdrawal amounts against prior deposits. A follow-up audit is strongly advised after implementing the recommended changes.

Issues Found

Severity	Number of Issues Found
High	8
Low	2
Total	10

Findings

High

[H-1] Users Who Give Tokens Approvals to L1BossBridge May Have Those Assets Stolen

Description:

The `L1BossBridge::depositTokensToL2` function allows anyone to call it with a `from` address of any account that has approved tokens to the bridge. As a consequence, an attacker can move tokens out of any victim account whose token allowance to the bridge is greater than zero. This will move the tokens into the bridge vault and assign them to the attacker's address in L2 (setting an attacker-controlled address in the `l2Recipient` parameter).

Impact:

The erroneous logic in `depositTokensToL2` allows attackers to steal other user's funds. Once the tokens are moved, the victim loses control of them. They are locked in the vault until a valid withdrawal signature is provided. While the attacker would need a signed message from an operator to withdraw, the immediate theft of the deposit credit and the loss of the token balance are irreversible without operator intervention. This violates the fundamental trust assumption that approving the bridge is safe and can drain any user who has ever approved the contract. The attack is trivial to execute and can be automated to sweep all approved allowances across all users, making it a high-severity risk.

Proof of Concept:

Consider the following scenario.

1. The user approves his tokens to be used by the `L1TokenBridge`.
2. An attacker monitors the transactions for `L1TokenBridge` and sees the approval transaction.
3. The attacker then invokes `depositTokensToL2`, stealing the user's funds.

Proof of Code

Insert the following test in `L1TokenBridge.t.sol`.

```
1 function testCanMoveApprovedTokensOfOtherUsers() public {
2     vm.prank(user);
3     token.approve(address(tokenBridge), type(uint256).max);
4     uint256 depositAmount = token.balanceOf(user);
5
6     address attacker = makeAddr("attacker");
7     vm.startPrank(attacker);
8     vm.expectEmit(address(tokenBridge));
9     emit Deposit(user, attacker, depositAmount);
10    tokenBridge.depositTokensToL2(user, attacker, depositAmount);
11    vm.stopPrank();
12
13    assertEq(token.balanceOf(user), 0);
14    assertEq(token.balanceOf(address(vault)), depositAmount);
15 }
```

Recommended Mitigation:

It is recommended to modify the `depositTokensToL2` function so that the caller cannot specify a `from` address.

```
1 - function depositTokensToL2(address from, address l2Recipient, uint256
   amount) external whenNotPaused {
2 + function depositTokensToL2(address l2Recipient, uint256 amount)
   external whenNotPaused {
3     if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
4         revert L1BossBridge__DepositLimitReached();
5     }
6 - token.transferFrom(from, address(vault), amount);
7 + token.transferFrom(msg.sender, address(vault), amount);
8
9     // Our off-chain service picks up this event and mints the
       corresponding tokens on L2
10 - emit Deposit(from, l2Recipient, amount);
11 + emit Deposit(msg.sender, l2Recipient, amount);
12 }
```

[H-2] Calling `L1BossBridge::depositTokensToL2` with the `from` and `to` Arguments Set to the Vault Contract Allows Infinite Minting of Unbacked Tokens

Description:

The `depositTokensToL2` function allows the caller to specify the `from` address, from which tokens are taken. Because the vault grants infinite approval to the bridge already (as can be seen in the contract's constructor), it is possible for an attacker to call the `depositTokensToL2` function and transfer tokens from the vault to the vault itself. As a result, this would allow the attacker to trigger the `Deposit` event any number of times, presumably causing the minting of unbacked tokens in L2. Additionally, they could mint all the tokens to themselves.

```
1     constructor(IERC20 _token) Ownable(msg.sender) {
2         token = _token;
3         vault = new L1Vault(token);
4         // Allows the bridge to move tokens out of the vault to
5         // facilitate withdrawals
6         @> vault.approveTo(address(this), type(uint256).max);
7     }
```

Impact:

This vulnerability allows an attacker to mint unlimited unbacked tokens on the L2 chain (ZKsync Era) without depositing any real assets into the bridge. The attack can be executed repeatedly, leading to a catastrophic protocol failure with multiple downstream consequences including: infinite unbacked minting, complete dilution of the L2 token supply, inability to reverse via burning illegitimate tokens, and permanent bridge insolvency.

The attacker can trigger the `Deposit` event an unlimited number of times by calling `depositTokensToL2(vault, attackerInL2, 0)`. Each call only costs gas and emits a `Deposit` event that the off-chain L2 service interprets as a legitimate deposit. The L2 service mints tokens to the attacker's address without any corresponding value locked on L1. Additionally, since the L2 tokens are minted without any backing collateral, the attacker can inflate the total supply arbitrarily, and thus devalue every legitimate token holder's position to zero. To add on, once the unbacked tokens are minted on L2, the attacker can swap them for other assets in L2 liquidity pools, extracting real value before the exploit is detected. The bridge has no mechanism to burn illegitimately minted tokens. Compounding these issues, when legitimate users eventually try to withdraw their L2 tokens back to L1 via the withdrawal mechanism, the vault will not contain enough tokens to honor all withdrawals. The bridge becomes insolvent, and any user with L2 tokens (whether legitimate or not) will be unable to redeem them for L1 assets. Ultimately, this results in the complete erosion of trust in the protocol: the trust required for the bridge to operate (L2 tokens are always 1:1 backed by L1 deposits) is permanently broken.

Proof of Concept:

Consider the following scenario.

1. Upon deployment of the bridge contract, the vault grants infinite approval to the bridge.
2. The attacker sees this attack vector and invokes `depositTokensToL2`, stealing the vault's funds continuously until it is depleted.

Proof of Code

Insert the following test in `L1TokenBridge.t.sol`.

```
1 function testCanTransferFromVaultToVault() public {
2     address attacker = makeAddr("attacker");
3     vm.startPrank(attacker);
4
5     // Assume the vault already holds some tokens
6     uint256 vaultBalance = 500 ether;
7     deal(address(token), address(vault), vaultBalance);
8
9     // Can trigger the `Deposit` event self-transferring tokens in the
    vault
10    vm.expectEmit(address(tokenBridge));
11    emit Deposit(address(vault), address(vault), vaultBalance);
12    tokenBridge.depositTokensToL2(address(vault), address(vault),
        vaultBalance);
13
14    // Any number of times
15    vm.expectEmit(address(tokenBridge));
16    emit Deposit(address(vault), address(vault), vaultBalance);
17    tokenBridge.depositTokensToL2(address(vault), address(vault),
        vaultBalance);
18
19    vm.stopPrank();
20 }
```

Recommended Mitigation:

Similar to H-1, it is recommended to modify the `depositTokensToL2` function so that the caller cannot specify a `from` address.

```
1 - function depositTokensToL2(address from, address l2Recipient, uint256
    amount) external whenNotPaused {
2 + function depositTokensToL2(address l2Recipient, uint256 amount)
    external whenNotPaused {
3     if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
4         revert L1BossBridge__DepositLimitReached();
5     }
6 - token.transferFrom(from, address(vault), amount);
7 + token.transferFrom(msg.sender, address(vault), amount);
8 }
```

```
9      // Our off-chain service picks up this event and mints the
      corresponding tokens on L2
10 -    emit Deposit(from, l2Recipient, amount);
11 +    emit Deposit(msg.sender, l2Recipient, amount);
12 }
```

[H-3] Lack of Replay Protection in L1BossBridge::withdrawTokensToL1 Allows Withdrawals by Signature to be Replayed

Description:

Users who wish to withdraw tokens from the bridge can call the `L1BossBridge::sendToL1` function or the wrapper `withdrawTokensToL1` function. These functions require the caller to send along some withdrawal data signed by one of the approved bridge operators. However, the signatures do not include any kind of replay-protection mechanism (e.g., nonces or a chain ID). As result, valid signatures from any bridge operator can be reused by any attacker to continue executing withdrawals until the vault is entirely drained of funds.

```
1      function withdrawTokensToL1(address to, uint256 amount, uint8 v,
      bytes32 r, bytes32 s) external {
2          sendToL1(
3              v,
4              r,
5              s,
6              abi.encode(
7                  address(token),
8                  0, // value
9                  abi.encodeCall(IERC20.transferFrom, (address(vault), to
      , amount))
10             )
11         );
12     }
13
14 @> function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory
      message) public nonReentrant whenNotPaused {
15     address signer = ECDSA.recover(MessageHashUtils.
      toEthSignedMessageHash(keccak256(message)), v, r, s);
16
17     if (!signers[signer]) {
18         revert L1BossBridge__Unauthorized();
19     }
20
21     (address target, uint256 value, bytes memory data) = abi.decode
      (message, (address, uint256, bytes));
22     (bool success,) = target.call{ value: value }(data);
23     if (!success) {
24         revert L1BossBridge__CallFailed();
25     }
26 }
```

```
25     }  
26 }
```

Impact:

An attacker who obtains any validly signed withdrawal message can replay it indefinitely to drain the vault of all tokens. The severity is amplified by four critical design flaws: no nonce, no chain ID, no deadline, and no recipient binding. First, the signature contains no unique identifier. Once a withdrawal is authorized, the same signature can be reused an unlimited number of times. Second, the signature is not bound to a specific chain. An attacker can take a valid signature from one chain and replay it on another chain where the same bridge is deployed, draining the vault there as well. Third, the signature has no expiry. A leaked or old signature remains valid forever, even if the signer's key is later rotated or revoked. Fourth, the signature authorizes a `to` address, but the attacker can substitute the recipient with their own address in the encoded message if the message structure allows it.

Proof of Concept:

Proof of Code

Insert the following test in `L1TokenBridge.t.sol`.

```
1  function testCanReplayWithdrawals() public {  
2      // Assume the vault already holds some tokens  
3      uint256 vaultInitialBalance = 1000e18;  
4      uint256 attackerInitialBalance = 100e18;  
5      deal(address(token), address(vault), vaultInitialBalance);  
6      deal(address(token), address(attacker), attackerInitialBalance);  
7  
8      // An attacker deposits tokens to L2  
9      vm.startPrank(attacker);  
10     token.approve(address(tokenBridge), type(uint256).max);  
11     tokenBridge.depositTokensToL2(attacker, attackerInitialBalance,  
12         attackerInitialBalance);  
13  
14     // Operator signs withdrawal.  
15     (uint8 v, bytes32 r, bytes32 s) =  
16         _signMessage(_getTokenWithdrawalMessage(attacker,  
17             attackerInitialBalance), operator.key);  
18  
19     // The attacker can reuse the signature and drain the vault.  
20     while (token.balanceOf(address(vault)) > 0) {  
21         tokenBridge.withdrawTokensToL1(attacker, attackerInitialBalance  
22             , v, r, s);  
23     }  
24     assertEq(token.balanceOf(address(attacker)), attackerInitialBalance  
25         + vaultInitialBalance);  
26     assertEq(token.balanceOf(address(vault)), 0);  
27 }
```

Recommended Mitigation:

It is recommended to redesign the withdrawal mechanism to include industry-standard replay protection. At a minimum, the signed message should include a nonce, chain ID, and deadline as shown below.

```
1 + // Track the next expected nonce for each signer
2 + mapping(address => uint256) public nonces;
3
4 + // Define a typed message for EIP-712 signatures (preferred) or at
   least include nonce in the message
5 + struct WithdrawalRequest {
6 +     address to;
7 +     uint256 amount;
8 +     uint256 nonce;
9 +     uint256 deadline;
10 +    uint256 chainId;
11 + }
12 + .
13 + .
14 + .
15 + function withdrawTokensToL1(
16 +     address to,
17 +     uint256 amount,
18 +     uint256 nonce,
19 +     uint256 deadline,
20 +     uint8 v,
21 +     bytes32 r,
22 +     bytes32 s
23 + ) external {
24 +     require(block.timestamp <= deadline, "Signature expired");
25
26 +     bytes32 messageHash = keccak256(
27 +         abi.encode(address(this), block.chainid, nonce, deadline,
28 +             to, amount)
29 +     );
30 +
31 +     address signer = ECDSA.recover(
32 +         MessageHashUtils.toEthSignedMessageHash(messageHash), v, r,
33 +         s
34 +     );
35
36 +     if (!signers[signer]) revert L1BossBridge__Unauthorized();
37 +
38 +     require(nonce == nonces[signer], "Invalid nonce");
39 +     nonces[signer]++;
40
41 +     sendToL1(
42 +         v,
43 +         r,
44 +         s,
```

```
43         abi.encode(  
44             address(token),  
45             0,  
46             abi.encodeCall(IERC20.transferFrom, (address(vault), to  
47                 , amount))  
48         );  
49     }
```

[H-4] L1BossBridge::sendToL1 Allowing Arbitrary Calls Enables Users to Call L1Vault::approveTo & Give Themselves Infinite Allowance of Vault Funds

Description:

The `L1BossBridge` contract includes the `sendToL1` function that, if called with a valid signature by an operator, can execute arbitrary low-level calls to any given target. Due to the lack of restrictions on both the target and calldata, this call could be used by an attacker to execute sensitive contracts of the bridge, such as the `L1Vault` contract.

Impact:

The `L1BossBridge` contract owns the `L1Vault` contract. Therefore, an attacker could submit a call that targets the vault and executes the `approveTo` function, passing an attacker-controlled address to increase its allowance. This would then allow the attacker to completely drain the vault.

It is worth noting that the likelihood of this attack depends on the level of sophistication of the off-chain validations implemented by the operators that approve and sign withdrawals. However, it is rated as a high severity issue because, according to the available documentation, the only validation made by off-chain services is that “the account submitting the withdrawal has first originated a successful deposit in the L1 part of the bridge”. As the provided proof of code demonstrates, such validation is not enough to prevent the attack.

Proof of Concept:

1. The attacker deposits tokens to the L2.
2. The attacker sends a message via `sendToL1` that instructs the vault to give an unlimited allowance.
3. The bridge operator does not validate the bytes being signed.
4. The vault grants the attacker infinite approval to the vault’s funds.
5. The attacker transfers out all tokens from the vault.

Proof of Code

Insert the following test in `L1TokenBridge.t.sol`.

```
1 function testCanCallVaultApproveFromBridgeAndDrainVault() public {
2     uint256 vaultInitialBalance = 1000e18;
3     deal(address(token), address(vault), vaultInitialBalance);
4
5     // An attacker deposits tokens to the L2
6     // Operator needs to see a valid deposit tx to then allow requests
7     // for a withdrawal
8     vm.startPrank(attacker);
9     vm.expectEmit(address(tokenBridge));
10    emit Deposit(address(attacker), address(0), 0);
11    tokenBridge.depositTokensToL2(attacker, address(0), 0);
12
13    // Assumes that the bridge operator doesn't validate bytes being
14    // signed
15    bytes memory message = abi.encode(
16        address(vault),
17        0,
18        abi.encodeCall(L1Vault.approveTo, (address(attacker), type(
19            uint256).max)) // data
20    );
21    (uint8 v, bytes32 r, bytes32 s) = _signMessage(message, operator.
22        key);
23
24    tokenBridge.sendToL1(v, r, s, message);
25    assertEq(token.allowance(address(vault), attacker), type(uint256).
26        max);
27    token.transferFrom(address(vault), attacker, token.balanceOf(
28        address(vault)));
29 }
```

Recommended Mitigation:

It is recommended to disallow attacker-controlled external calls to sensitive components of the bridge, such as the `L1Vault` contract.

```
1 function sendToL1(uint8 v, bytes32 r, bytes32 s, address to,
2     uint256 amount) public nonReentrant whenNotPaused {
3     +     bytes memory message = abi.encode(
4     +         address(token),
5     +         0,
6     +         abi.encodeCall(IERC20.transferFrom, (address(vault), to,
7     +             amount))
8     +     );
9
10    address signer = ECDSA.recover(MessageHashUtils.
11        toEthSignedMessageHash(keccak256(message)), v, r, s);
12
13    if (!signers[signer]) {
14        revert L1BossBridge__Unauthorized();
15    }
16 }
```

```
13
14 -     (address target, uint256 value, bytes memory data) = abi.decode
      (message, (address, uint256, bytes));
15 -     (bool success,) = target.call{ value: value }(data);
16 -     if (!success) {
17 -         revert L1BossBridge__CallFailed();
18 -     }
19 +     token.safeTransferFrom(address(vault), to, amount);
20 }
```

[H-5] Assembly CREATE Usage in TokenFactory::deployToken Is Incompatible with ZKsync Native EraVM

Description:

The `TokenFactory::deployToken` function deploys new ERC20 contracts using the `create` opcode in inline assembly.

```
1     function deployToken(string memory symbol, bytes memory
      contractBytecode) public onlyOwner returns (address addr) {
2         assembly {
3 @>         addr := create(0, add(contractBytecode, 0x20), mload(
      contractBytecode))
4         }
5         s_tokenToAddress[symbol] = addr;
6         emit TokenDeployed(symbol, addr);
7     }
```

ZKsync's native EraVM does not support deploying contracts from arbitrary bytecode using `CREATE` in assembly blocks. The ZKsync documentation explicitly warns that this pattern “will never work because contracts cannot be deployed on EraVM using any kind of bytecode” and that the compiler “will most likely silently produce broken bytecode.”

While the newer EVM Bytecode Interpreter supports `CREATE/CREATE2` for standard EVM contracts deployed via the new keyword, assembly-level deployment of arbitrary bytecode remains incompatible with EraVM's deployment model, which uses bytecode hashes and the `ContractDeployer` system contract rather than raw bytecode.

Impact:

The `deployToken` function will either revert or produce broken, unusable contract deployments when compiled with `zksolc` for native EraVM execution. The factory becomes non-functional on ZKsync chains, permanently breaking the protocol's token deployment capability. Since `s_tokenToAddress[symbol]` is set to the (potentially invalid) returned address, the mapping may store a broken reference that cannot be corrected.

Proof of Concept:

View the ZKsync Era documentation on EVM instruction differences: <https://docs.zksync.io/zksync-protocol/era-vm/differences/evm-instructions#create-create2>

The pattern used in the `deployToken` function (`create(0, add(bytecode, 0x20), mload(bytecode))`) with arbitrary bytecode passed as a parameter is exactly the pattern that the documentation explicitly states “will never work.”

Recommended Mitigation:

It is recommended to replace the assembly `create` with standard Solidity `new` syntax. This approach is safe on both the EVM Bytecode Interpreter and native EraVM, as the compiler can correctly handle the deployment through the proper system contract.

```
1 - function deployToken(string memory symbol, bytes memory
   contractBytecode) public onlyOwner returns (address addr) {
2 + function deployToken(string memory symbol) public onlyOwner returns
   (address addr) {
3 -     assembly {
4 -         addr := create(0, add(contractBytecode, 0x20), mload(
           contractBytecode))
5 -     }
6 +     L1Token token = new L1Token();
7 +     addr = address(token);
8     s_tokenToAddress[symbol] = addr;
9     emit TokenDeployed(symbol, addr);
10 }
```

[H-6] L1BossBridge::DEPOSIT_LIMIT Check in L1BossBridge::depositTokensToL2 Combined with Front-Running Enables Permanent DoS of Deposits**Description:**

The `depositTokensToL2` function enforces a fixed, non-adjustable deposit limit by checking:

```
1 if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
2     revert L1BossBridge__DepositLimitReached();
3 }
```

Since `DEPOSIT_LIMIT` is a hardcoded value (`100_000 ether`) with no mechanism to update it, an attacker can deliberately fill the vault to the limit. Once the limit is reached, the condition will revert every subsequent call to `depositTokensToL2`, even for small, legitimate deposits. The only way to restore deposit functionality is to withdraw tokens from the vault, which requires a valid signature from an authorized signer.

Impact:

An attacker can exploit this attack vector by:

1. Depositing enough tokens to push the vault balance to `DEPOSIT_LIMIT` (or just below it).
2. Monitoring the mempool for incoming deposit transactions and front-running them with a tiny deposit that breaches the limit.

All future deposits will then revert until a withdrawal is processed. This can result in complete denial of service, rendering the bridge's deposit function unusable for all honest users, permanently (or until the attacker decides to withdraw). Note that the attacker must lock up a large amount of tokens, but this capital is not lost (only locked up which makes the attack nearly cost-free).

Proof of Concept:

Consider the following scenario.

1. The attacker has been granted approval and tokens.
2. The attacker deposits exactly the limit or just under the limit.
3. Any legitimate users will now be blocked from using the protocol.

Proof of Code

Insert the following test into `L1TokenBridge.t.sol`.

```
1     function testDosByFillingDepositLimit() public {
2         // Set up attacker
3         address attacker = makeAddr("attacker");
4         uint256 limit = tokenBridge.DEPOSIT_LIMIT();
5         deal(address(token), attacker, limit);
6         vm.startPrank(attacker);
7         token.approve(address(tokenBridge), limit);
8
9         // Deposit exactly the limit so vault balance == DEPOSIT_LIMIT
10        tokenBridge.depositTokensToL2(attacker, attacker, limit);
11        vm.stopPrank();
12
13        // Now any further deposit, even 1 wei, will revert
14        vm.startPrank(user);
15        token.approve(address(tokenBridge), 1 ether);
16        vm.expectRevert(L1BossBridge.L1BossBridge__DepositLimitReached.selector);
17        tokenBridge.depositTokensToL2(user, user, 1 ether);
18        vm.stopPrank();
19    }
```

Recommended Mitigation:

It is recommended to consider making one or more of the following changes:

1. Make the limit adjustable: add a function that uses the `onlyOwner` modifier to update `DEPOSIT_LIMIT`, so the protocol can react if an attack occurs.
2. Remove the global limit entirely: if the purpose is to prevent infinite minting, rely on L2 supply controls rather than a hard L1 cap.
3. Implement a rate-limiting mechanism: instead of a global balance ceiling, enforce per-account or per-time-window deposit limits that are less susceptible to single-actor front-running.

At minimum, allow the owner to raise or remove the limit via a privileged function:

```
1 + function setDepositLimit(uint256 newLimit) external onlyOwner {  
2 +     DEPOSIT_LIMIT = newLimit;  
3 + }
```

[H-7] Missing Validation in `L1BossBridge::withdrawTokensToL1` on the Withdrawal Amount Allows Attackers to Withdraw More Tokens Than Deposited

The `L1BossBridge::depositTokensToL2` function allows users to deposit tokens into the vault and receive a corresponding amount of tokens on the L2. The `withdrawTokensToL1` function (and the underlying `L1BossBridge::sendToL1`) is responsible for the reverse flow: burning tokens on the L2 and releasing tokens from the vault on the L1. However, there is no on-chain tracking of the amount of tokens a user has deposited. The `depositTokensToL2` function emits a `Deposit` event but does not store the deposit amount in any state variable.

```
1 function depositTokensToL2(address from, address l2Recipient,  
2   uint256 amount) external whenNotPaused {  
3   if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {  
4       revert L1BossBridge__DepositLimitReached();  
5   }  
6   token.safeTransferFrom(from, address(vault), amount);  
7   @> emit Deposit(from, l2Recipient, amount);  
8 }
```

The withdrawal flow relies entirely on off-chain validation by the operator, who is supposed to verify that the user has deposited enough tokens before signing a withdrawal message. There is no on-chain invariant enforcing that `withdrawAmount` $\leq$ `depositedAmount`.

Impact:

This withdrawal amount issue results in several impacts. For one, an attacker who deposits 1 token can request a withdrawal for 1000 tokens. If the operator signs the message without proper validation (or if the attacker compromises an operator key), the vault will release far more tokens than the attacker ever deposited. Furthermore, legitimate depositors whose L2 tokens are backed by real L1 deposits will find the vault empty when they try to withdraw, rendering their L2 tokens worthless. In essence,

the entire security of the withdrawal mechanism is dependent on the operator's off-chain validation. Any bug, social engineering attack, or key compromise in the operator's infrastructure leads directly to fund loss.

Proof of Concept:

Consider the following scenario.

1. The attacker deposits a 1 token.
2. The operator signs a withdrawal for the attacker.
3. The attacker is able to withdraw more than he has deposited (e.g., 1000 tokens).

Proof of Code

Insert the following test into `L1TokenBridge.t.sol`.

```
1      function testCanWithdrawMoreThanDeposited() public {
2          // Attacker deposits a small amount
3          address attacker = makeAddr("attacker");
4          uint256 depositAmount = 1e18;
5          uint256 withdrawAmount = 1000e18;
6
7          deal(address(token), attacker, depositAmount);
8          vm.startPrank(attacker);
9          token.approve(address(tokenBridge), type(uint256).max);
10         tokenBridge.depositTokensToL2(attacker, attacker, depositAmount
11             );
12         vm.stopPrank();
13
14         // Fund the vault so there are tokens to steal
15         deal(address(token), address(vault), withdrawAmount);
16
17         // Operator signs a withdrawal for 1000x the deposited amount
18         (uint8 v, bytes32 r, bytes32 s) =
19             _signMessage(_getTokenWithdrawalMessage(attacker,
20                 withdrawAmount), operator.key);
21
22         // Attacker withdraws 1000x more than they deposited
23         uint256 attackerBalanceBefore = token.balanceOf(attacker);
24         vm.prank(attacker);
25         tokenBridge.withdrawTokensToL1(attacker, withdrawAmount, v, r,
26             s);
27
28         assertEq(token.balanceOf(attacker), attackerBalanceBefore +
29             withdrawAmount);
30         assertGt(token.balanceOf(attacker), depositAmount);
31     }
```

Recommended Mitigation:

It is recommended to add on-chain deposit tracking. One way to do this is to maintain a mapping of each user's deposited balance and enforce that withdrawals cannot exceed deposits.

```
1 + mapping(address => uint256) private depositedBalance;
2
3 function depositTokensToL2(address from, address l2Recipient,
4   uint256 amount) external whenNotPaused {
5     if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
6       revert L1BossBridge__DepositLimitReached();
7     }
8 +   depositedBalance[from] += amount;
9   token.safeTransferFrom(from, address(vault), amount);
10  emit Deposit(from, l2Recipient, amount);
11 }
12
13 function withdrawTokensToL1(address to, uint256 amount, uint8 v,
14   bytes32 r, bytes32 s) external {
15 +   require(amount <= depositedBalance[to], "Withdrawal exceeds
16   deposit");
17 +   depositedBalance[to] -= amount;
18   sendToL1(
19     v, r, s,
20     abi.encode(
21       address(token), 0,
22       abi.encodeCall(IERC20.transferFrom, (address(vault), to
23         , amount))
24     )
25   );
26 }
```

[H-8] TokenFactory::deployToken Causes All Minted Tokens to Be Permanently Locked in the Factory Contract

Description:

The `TokenFactory` contract deploys new ERC20 tokens using the `create` opcode. Within the `L1Token` constructor, the initial supply is minted to `msg.sender`:

```
1 constructor() ERC20("BossBridgeToken", "BBT") {
2   _mint(msg.sender, INITIAL_SUPPLY * 10 ** decimals());
3 }
```

Since the deployment is carried out through the `create` opcode inside the factory, `msg.sender` during the token construction is the factory itself. Therefore, all newly minted tokens are sent to the factory's address. However, the factory has no function to transfer, withdraw, or otherwise handle ERC20 tokens, and the `Ownable` owner has no mechanism to rescue them. As a consequence of this, the tokens become permanently inaccessible.

Impact:

This improper handling of the `BossBridgeToken` ERC20 causes a two-fold impact. First, the entire initial supply of each deployed token is irretrievably locked in the factory contract. Second, the intended recipients (e.g., users, bridging contracts) never receive the tokens, which renders the deployed token contracts useless. This is exacerbated by the fact that the factory lacks a rescue function and ownership does not confer the ability to move tokens held by the contract.

Proof of Concept:

The following sequence demonstrates the issue:

1. The owner calls `TokenFactory.deployToken("BBT", l1TokenBytecode)`.
2. The create opcode deploys the `L1Token` contract.
3. During construction, `L1Token._mint(msg.sender, ...)` mints the entire supply to the factory address.
4. After deployment, `L1Token.balanceOf(address(factory))` equals the total initial supply.
5. No function in the factory can transfer these tokens, and the owner cannot retrieve them.

Recommended Mitigation:

It is recommended to modify the token constructor to accept a `_recipient` parameter and mint to that address instead of `msg.sender`. When deploying from the factory, pass the intended recipient as a constructor argument.

```
1 - constructor() ERC20("BossBridgeToken", "BBT") {
2 + constructor(address _recipient) ERC20("BossBridgeToken", "BBT") {
3 -     _mint(msg.sender, INITIAL_SUPPLY * 10 ** decimals());
4 +     _mint(_recipient, INITIAL_SUPPLY * 10 ** decimals());
5 }
```

Low**[L-1] Missing Events on Withdrawal Functions Obscure Off-Chain Monitoring and Incident Response****Description:**

The L1 side of the Boss Bridge emits a `Deposit` event when users deposit tokens to L2, but neither the `L1BossBridge::sendToL1` function nor the `L1BossBridge::withdrawTokensToL1` function emit an event when a withdrawal is successfully executed.

```
1     function depositTokensToL2(address from, address l2Recipient,
2         uint256 amount) external whenNotPaused {
3         // ... deposit tokens ...
4         emit Deposit(from, l2Recipient, amount);
5     }
6
7     function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory
8         message) public nonReentrant whenNotPaused {
9         // ... signature verification, call execution ...
10        // No event emitted
11    }
12
13    function withdrawTokensToL1(address to, uint256 amount, uint8 v,
14        bytes32 r, bytes32 s) external {
15        // ... token transfer ...
16        // No event emitted
17    }
```

Impact:

The lack of event emission in the `L1BossBridge` contract results in several impacts. It creates a monitoring gap, preventing off-chain indexers, security alerting systems, and analytics dashboards from reliably tracking withdrawals in real time. This delays detection of anomalous or unauthorized withdrawals until after the fact, which potentially allows malicious activity to go undetected for longer periods. Additionally, this creates operational blind spots in which the protocol operators and users lack a complete view of the bridge's asset flow, complicating reconciliation as well as incident response.

Proof of Concept:

1. A user deposits tokens via `L1BossBridge::depositTokensToL2` and a `Deposit` event is correctly emitted and can be observed off-chain.
2. The same user later initiates a withdrawal via `withdrawTokensToL1` or `sendToL1`. The withdrawal succeeds, but no event is emitted.
3. An off-chain monitoring service scanning for `Deposit` events alone cannot correlate them with subsequent withdrawals, causing an incomplete view of the bridge's activity.

Recommended Mitigation:

It is recommended to introduce a dedicated event and emit it at the end of each successful withdrawal.

```
1 +     event Withdrawal(address indexed to, uint256 amount);
2 +     event L1Transfer(address indexed target, uint256 value, bytes data)
3       ;
4
5
```

```
6     function withdrawTokensToL1(address to, uint256 amount, uint8 v,
7         bytes32 r, bytes32 s) external {
8 +     // ... existing logic ...
9         emit Withdrawal(to, amount);
10    }
11    function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory
12        message) public nonReentrant whenNotPaused {
13        // ... existing logic ...
14        (address target, uint256 value, bytes memory data) = abi.decode
15            (message, (address, uint256, bytes));
16        emit L1Transfer(target, value, data);
17    }
```

[L-2] TokenFactory::deployToken Can Create Multiple Tokens with the Same symbol

Description:

The `deployToken` function has no checks in place to prevent a new token address from overwriting a previous token address with the same symbol.

```
1     function deployToken(string memory symbol, bytes memory
2         contractBytecode) public onlyOwner returns (address addr) {
3         assembly {
4             addr := create(0, add(contractBytecode, 0x20), mload(
5                 contractBytecode))
6         }
7         s_tokenToAddress[symbol] = addr;
8         emit TokenDeployed(symbol, addr);
9     }
```

Impact:

If the deployment team is not meticulous with their Boss Bridge system token naming conventions, they can overwrite an existing token address. To elaborate, if the token address for a given symbol is overwritten, any off-chain processes, bridge components, or on-chain contracts that rely on `TokenFactory::getTokenAddressFromSymbol` will point to the new token instead of the original, potentially disrupting the entire bridge ecosystem until corrected manually.

Proof of Concept:

Consider the following scenario.

1. The protocol team deploys a token named “Boss Bridge Token A” with the symbol “BBT”.
2. The Boss Bridge Token A token with the symbol “BBT” gets stored in the `s_tokenToAddress` mapping.

3. Later, the protocol team deploys another token named “Boss Bridge Token B” with the same symbol of “BBT”.
4. The address of the Boss Bridge Token A token is overwritten in the `s_tokenToAddress` mapping.

Proof of Code

Insert the following test into `TokenFactorTest.t.sol`.

```
1     function testAddDuplicateSymbolToken() public {
2         vm.startPrank(owner);
3         // Create Boss Bridge Token A with symbol "BBT"
4         address tokenAAddress = tokenFactory.deployToken("BBT", type(
5             L1Token).creationCode);
6         assertEq(tokenFactory.getTokenAddressFromSymbol("BBT"),
7             tokenAAddress);
8         // Create Boss Bridge Token B with symbol "BBT"
9         address tokenBAddress = tokenFactory.deployToken("BBT", type(
10            L1Token).creationCode);
11        // Assert s_tokenToAddress["BBT"] == tokenBAddress
12        assertNotEq(tokenFactory.getTokenAddressFromSymbol("BBT"),
13            tokenAAddress);
14        assertEq(tokenFactory.getTokenAddressFromSymbol("BBT"),
15            tokenBAddress);
16        vm.stopPrank();
17    }
```

Recommended Mitigation:

It is recommended to add a check before updating the `s_tokenToAddress` mapping.

```
1 +     error TokenFactory__TokenSymbolAlreadyExists();
2     .
3     .
4     .
5     function deployToken(string memory symbol, bytes memory
6         contractBytecode) public onlyOwner returns (address addr) {
7         assembly {
8             addr := create(0, add(contractBytecode, 0x20), mload(
9                 contractBytecode))
10        }
11 +     if (s_tokenToAddress[symbol] != address(0)) revert
12        TokenFactory__TokenSymbolAlreadyExists();
13        s_tokenToAddress[symbol] = addr;
14        emit TokenDeployed(symbol, addr);
15    }
```